

Dokumentacja procesu zasilania hurtowni danych (ELT)

Spis treści

1. Architektura procesu	3
2. Opis zadań	4
Faza 1: Extract & Load	4
Faza 2: Transform – pakiet staging	5
Faza 3: Transform – pakiet wymiarów	5
Faza 4: Transform – pakiet faktów	6
Faza 5: Jakość danych (automatyczne testy)	6
3. Model fizyczny a logiczny (diagramy tabel)	7

1. Architektura procesu

Proces zasilania hurtowni danych został zrealizowany w paradygmacie **ELT (Extract, Load, Transform)**.

Dlaczego wybrano ELT zamiast tradycyjnego ETL?

W klasycznym podejściu ETL, transformacja danych odbywa się w pamięci operacyjnej pośredniczącego serwera integracyjnego, co przy pracy z wielomilionowymi zbiorami danych (takimi jak loty w USA) stwarza wąskie gardło wydajnościowe. Podejście ELT eliminuje ten problem – surowe dane są najpierw błyskawicznie ładowane do bazy, a następnie transformowane bezpośrednio w niej. Dzięki temu proces w pełni wykorzystuje potężną moc obliczeniową docelowego silnika relacyjnego (SQL Server).

Przechowywanie tych dodatkowych, surowych danych wewnątrz bazy nie jest dziś żadnym problemem, ponieważ przestrzeń dyskowa jest bardzo tania. Wielkim plusem takiego podejścia jest to, że mamy w bazie stały dostęp do pełnych, nietkniętych danych źródłowych, do których zawsze możemy wrócić w przypadku błędu lub zmiany wymagań analitycznych.

Zamiast graficznych narzędzi, zastosowano podejście *Data as Code*, wykorzystując:

- **skrypty** w języku **Python**: do ekstrakcji, budowy słowników referencyjnych oraz ładowania.
- framework **dbt (Data Build Tool)**: do transformacji wewnątrz silnika bazy danych SQL Server.
- **makra**: dbt pozwala na tworzenie reużywalnych bloków kodu. W projekcie wykorzystano makro `clean_tail_number.sql`, które dynamicznie standaryzuje numery rejestracyjne (np. inteligentnie doklejając brakujący prefiks 'N').
- słowniki statyczne (**seeds**): pliki CSV (np. `airlines.csv`, wygenerowany `mfr_mapping.csv`), które dbt automatycznie ładuje do bazy jako tabele referencyjne.

Całością orkiestruje nadrzędny skrypt wsadowy (`run_elt.bat`).



Mapa logiczna procesu ELT (Directed Acyclic Graph wygenerowany przez dbt)

2. Opis zadań

Całość procesu jest zorganizowana w architekturze wielowarstwowej – proces został podzielony na wyizolowane logicznie pakiety (fazy).

Jego głównymi warstwami są **raw**, **staging** i **core**. Dlaczego taki podział?

Celem jest uniknięcie bałaganu w kodzie i trzymanie się zasady *separation of concerns* (rozdziatu obowiązków). Taki podział zabezpiecza hurtownię przed błędami.

- **Warstwa raw:** Przechowuje zrzut 1:1 tego, co przyszło w plikach źródłowych.
- **Warstwa staging:** Działa jak filtr. Czyści błędy techniczne (ucina spację, naprawia zepsute formaty dat i typuje kolumny), ale nie rusza logiki biznesowej.
- **Warstwa core (wymiary i fakty):** Otrzymuje już ładnie przygotowane i wyczyszczone dane ze stagingu. Dzięki temu może skupić się wyłącznie na tym, co najważniejsze: hashowaniu kluczy, łączeniu tabel i przeliczaniu reguł operacyjnych.

Co niezwykle ważne, taki podział jest bardzo wydajny i eliminuje powielanie pracy. Wiele docelowych tabel w warstwie core korzysta z tego samego źródła (np. `dim_date`, `dim_time`, i `fact_flights` czerpią z lotów). Zamiast w każdej z tych tabel od nowa pisać instrukcje naprawiające format czasu, wykonujemy to tylko raz – na etapie stagingu. Potem warstwa core po prostu korzysta z tego jednego, wspólnego i czystego fundamentu.

Poniżej przedstawiono szczegółowy opis poszczególnych faz procesu:

Faza 1: Extract & Load

Faza ta przetwarza pliki przed ich wejściem do hurtowni, skupiając się na budowie słowników oraz optymalnym ładowaniu do warstwy raw.

- **Zadanie: `generate_mfr_mapping.py`**
 - **Cel i działanie:** Ujednolicenie tysięcy wariacji nazw producentów samolotów (np. „BOEING INC”, „BOEING CO”) do wspólnego standardu.
Skrypt realizuje mapowanie w 4 krokach:
 1. Standaryzacja źródłowa: Od razu przy wczytywaniu danych skrypt wymusza wielkie litery i ucinanie ukrytych spacji. Zapobiega to późniejszemu problemowi duplikowania się kluczy przy łączeniu tabel w warstwie SQL.
 2. Reguły twarde: Wychwytywanie głównych graczy (Airbus, Boeing) przy pomocy list słów kluczowych i wyrażeń regularnych.
 3. Czyszczenie tekstu: Usuwanie przyrostków typu INC, LLC oraz znaków specjalnych i interpunkcji.
 4. Algorytm ML (Fuzzy Matching): Reszta nieznanych nazw trafia do algorytmu NLP. Skrypt wektoryzuje teksty znakowo (TF-IDF), a następnie przez szybką miarę podobieństwa cosinusowego namierza potencjalne klastry słów. Ostateczna decyzja o połączeniu firm w jedną nazwę jest podejmowana na podstawie algorytmu odległości Levenshteina. Algorytm grupuje nazwy o podobieństwie powyżej 80%.
 - **Wynik:** Wygenerowany słownik `mfr_mapping.csv`, który trafia do modułu `seed`.

```
Rozpoczynam generowanie słownika producentow...
Zbudowano globalną listę do analizy: 44196 unikalnych nazw.
Złapano 933 wariacji regułami. Zostało 43263 nazw dla ML...
Unikalnych producentów po ML: 37800
-> Zakończono! Całkowity czas: 490.41 sek.
```

- **Zadanie: load_raw_data.py**

- *Cel i działanie:* Ładowanie danych do bazy. Aby uniknąć zapchania pamięci RAM przy pliku ważącym blisko 600 MB (ponad 5 milionów lotów), zastosowano metodę paczkowania danych (tzw. chunking, porcje po 150 tys. wierszy).

Faza 2: Transform – pakiet staging

Celem zadań stagingowych jest techniczne „posprzątanie” surowych danych, bez ingerencji w logikę biznesową.

- **Zadanie: stg_flights**

- Rozwiązuje problem zepsutego formatu systemowego godzin (np. zamienia wartość 630 na prawidłową godzinę 6 i minutę 30).
- Wywołuje przygotowane wcześniej makro clean_tail_number, aby inteligentnie dokleić brakujące prefiksy 'N' do numerów rejestracyjnych maszyn.
- Rzuca kilkanaście kolumn (dystans, opóźnienia) na ścisłe typy liczbowe.

- **Zadanie: stg_aircraft_db oraz stg_aircraft_faa**

- Zabezpieczają kolejne warstwy przed ułomnościami zewnętrznych baz samolotów. Zamieniają fałszywe zera w liczbie siedzeń lub nieprawidłowe daty na formalne wartości NULL.
- Wykonują złączenie ze słownikiem mfr_mapping, podmieniając surowe nazwy producentów na wersje ustandaryzowane.
- stg_aircraft_db realizuje wstępną deduplikację wpisów historycznych po numerze rejestracyjnym z wykorzystaniem funkcji okna.

- **Zadanie: stg_airports**

- Wykonuje bazowe czyszczenie białych znaków (LTRIM/RTRIM) z pól opisowych oraz rzutowanie typów.

Faza 3: Transform – pakiet wymiarów

Warstwa tworząca zdenormalizowane tabele wymiarów. Tworzą sztuczne klucze główne (SK) oraz dołączają wiersz techniczny UNKNOWN chroniący przed utratą faktów operacyjnych z powodu błędów dopasowania kluczy.

- **Zadanie: dim_airport**

- Tworzy sztuczny klucz Airport_SK na podstawie trzyliterowego kodu IATA, wykorzystując do tego algorytm MD5. Wyciąga podstawowe informacje geograficzne (miasto, stan i państwo).

- **Zadanie: dim_aircraft**

- Tworzy jeden zintegrowany słownik floty. Łączy dane z FAA i aircraftDB za pomocą FULL OUTER JOIN, ucinając duplikaty atrybutów funkcją COALESCE. Tutaj też budowane są przedziały kategoryzujące dla niektórych atrybutów (np. grupowanie wielkości maszyn na podstawie liczby miejsc).

- **Zadanie: dim_airline oraz dim_cancellation_reason**

- Zamieniają systemowe, mało czytelne kody na wartości biznesowe (np. klasyfikują konkretne linie jako „Low-Cost Carrier” lub tłumaczą powód odwołania z litery „B” na „Weather”).

- **Zadania: dim_date oraz dim_time**

- Zwykle elementy daty są tu wzbogacane o atrybuty takie jak kwartał, pora roku, flaga weekendu czy pora dnia.

Faza 4: Transform – pakiet faktów

To centralny punkt hurtowni – zadanie agreguje główne miary lotów i buduje tabelę faktów oraz cały schemat poprzez złączenie z tabelami wymiarów.

- **Budowa schematu gwiazdy:** Przez operacje relacyjne lub obliczanie hashy, podłączane są wymiary w oparciu o ich klucze.
- **Reguły departamentu transportu USA:** Oczyszczane są miary – za pomocą logiki warunkowej pilnujemy, aby lot odwołany lub przekierowany miał wymuszone wartości NULL dla czasu kołowania i miar opóźnień. Ponadto dla lotów punktualnych (całkowite opóźnienie mniejsze niż 15 min), system wymusza zera w podkategoriach opóźnień zamiast wartości NULL.
- **Ładowanie przyrostowe:** Tabela działa w trybie dopisywania (append). W oparciu o wyliczony Flight_Business_Key z atrybutów lotu, dbt dopisuje tylko nowe loty i nie musi przetwarzać milionów historycznych wierszy przy każdym uruchomieniu potoku.

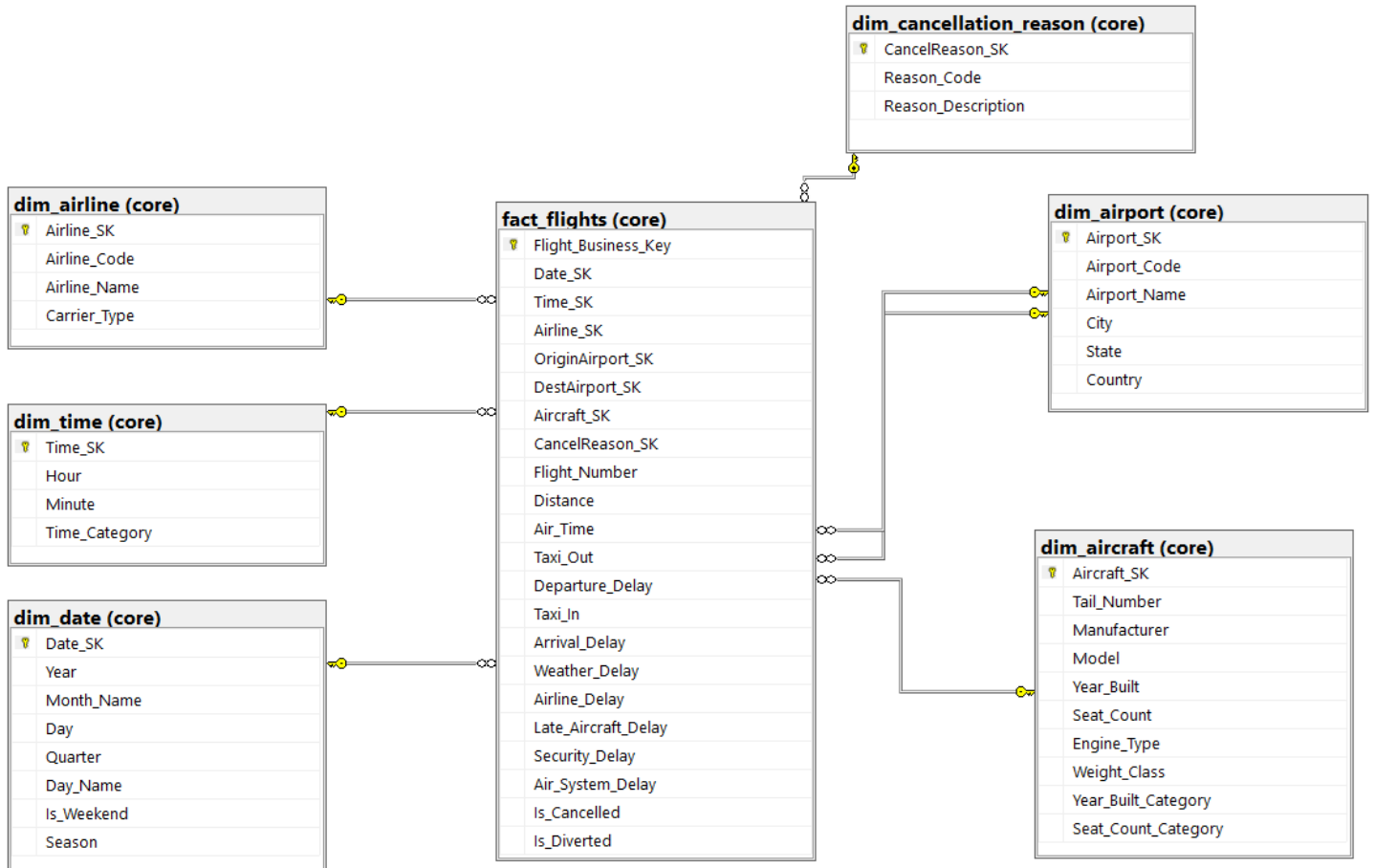
Faza 5: Jakość danych (automatyczne testy)

Sprawdzanie poprawności danych w finalnych tabelach wymiarów i faktów wykonano poprzez zautomatyzowany moduł QA w dbt. Jeśli dane złamią zdefiniowane reguły, dostaniemy dokładne informacje.

1. **Testy generyczne:** Weryfikują unikalność kluczy (unique), brak wartości pustych (not_null), akceptowalne wartości oraz integralność referencyjną relationships (sprawdzają, czy klucz obcy w faktach faktycznie istnieje w wymiarze).
2. **Testy logiczne:** Osobne skrypty SQL sprawdzające założenia biznesowe. Pilnują one m.in. braku miar dla lotów przekierowanych, dodatniego przebytego dystansu oraz matematycznego zbilansowania – upewniają się, że suma 5 drobnych opóźnień dla danego lotu równa się dokładnie całkowitemu opóźnieniu w przylocie itp.

3. Model fizyczny a logiczny (diagramy tabel)

W ramach wizualizacji architektury, przygotowano zrzuty ekranu ze środowiska SQL Server Management Studio (SSMS).



Logiczny model danych (ze zdefiniowanymi relacjami) Przedstawia docelowy schemat gwiazdy, uwytatniajacy relacje powiazan (PK/FK) pomiedzy centralna tabelą faktów a otaczajacymi ją wymiarami. Pełni on rolę konceptu analitycznego pokazujacego co i jak ze sobą współpracuje.

dim_airport (core)

Nazwa kolumny	Typ skondensowany
Airport_SK	uniqueidentifier
Airport_Code	varchar(7)
Airport_Name	varchar(80)
City	varchar(40)
State	varchar(7)
Country	varchar(7)

dim_time (core)

Nazwa kolumny	Typ skondensowany
Time_SK	int
Hour	int
Minute	int
Time_Category	varchar(9)

dim_date (core)

Nazwa kolumny	Typ skondensowany
Date_SK	int
Year	int
Month_Name	nvarchar(30)
Day	int
Quarter	int
Day_Name	nvarchar(30)
Is_Weekend	int
Season	varchar(6)

fact_flights (core)

Nazwa kolumny	Typ skondensowany
Flight_Business...	varchar(59)
Date_SK	int
Time_SK	int
Airline_SK	uniqueidentifier
OriginAirport_SK	uniqueidentifier
DestAirport_SK	uniqueidentifier
Aircraft_SK	uniqueidentifier
CancelReason_...	uniqueidentifier
Flight_Number	varchar(10)
Distance	int
Air_Time	decimal(10, 2)
Taxi_Out	decimal(10, 2)
Departure_Delay	decimal(10, 2)
Taxi_In	decimal(10, 2)
Arrival_Delay	decimal(10, 2)
Weather_Delay	decimal(10, 2)
Airline_Delay	decimal(10, 2)
Late_Aircraft_D...	decimal(10, 2)
Security_Delay	decimal(10, 2)
Air_System_Del...	decimal(10, 2)
Is_Cancelled	bit
Is_Diverted	bit

dim_cancellation_reason (core)

Nazwa kolumny	Typ skondensowany
CancelReason_...	uniqueidentifier
Reason_Code	varchar(1)
Reason_Descri...	varchar(19)

dim_airline (core)

Nazwa kolumny	Typ skondensowany
Airline_SK	uniqueidentifier
Airline_Code	varchar(16)
Airline_Name	varchar(28)
Carrier_Type	varchar(22)

dim_aircraft (core)

Nazwa kolumny	Typ skondensowany
Aircraft_SK	uniqueidentifier
Tail_Number	varchar(10)
Manufacturer	varchar(30)
Model	varchar(20)
Year_Built	int
Seat_Count	int
Engine_Type	varchar(13)
Weight_Class	varchar(19)
Year_Built_Cate...	varchar(18)
Seat_Count_Cat...	varchar(26)

Fizyczny model danych (implementacja produkcyjna) Przedstawia faktyczny, docelowy stan wygenerowany przez dbt wraz z typami danych. W nowoczesnych systemach analitycznych celowo rezygnuje się z fizycznych, twardych więzów integralności – constraintów FK. Narzucenie takich ograniczeń zmuszałoby silnik SQL Server do ciężkiej walidacji relacyjnej przy próbie zapisu każdej operacji lotniczej, co całkowicie zabiłoby wydajność ładowania. Spójność zapewniamy logicznie – poprzez testy relationships w dbt, daje nam to lepszą wydajność procesowania.